

Te he preparado un manual en formato Markdown ([solucion-docker-mysql.md](#)) que puedes guardar en tu proyecto y consultar cada vez que vuelva a aparecer alguno de estos problemas. Cubre:

- Conflicto con MariaDB local (puerto 3306 ocupado).
- Error de acceso denegado al usuario `erp_user` (Error 1045).
- Contenedor `erp_mysql` no arranca o queda en estado "unhealthy".
- phpMyAdmin falla porque MySQL no está sano.
- Cambio de puerto para evitar colisiones futuras.

Manual de resolución de problemas – Docker + MySQL + FastAPI

Este documento recoge los errores más comunes al levantar el entorno Docker (MySQL, Redis, phpMyAdmin) junto con el backend FastAPI y cómo solucionarlos de forma definitiva.

- [1. El puerto 3306 ya está en uso (conflicto con MariaDB local)](#1-el-puerto-3306-ya-está-en-uso-conflicto-con-mariadb-local)
- [2. Error 1045 – Access denied for user 'erp_user'@'localhost'](#2-error-1045--access-denied-for-user-erp_userlocalhost)
- [3. El contenedor `erp_mysql` no se considera healthy o tarda en arrancar] (#3-el-contenedor-erp_mysql-no-se-considera-healthy-o-tarda-en-arrancar)
- [4. phpMyAdmin no inicia porque MySQL no está healthy] (#4-phpmyadmin-no-inicia-porque-mysql-no-está-healthy)
- [5. Cambio permanente de puerto para evitar conflictos con MariaDB] (#5-cambio-permanente-de-puerto-para-evitar-conflictos-con-mariadb)
- [6. Comandos de diagnóstico rápido] (#6-comandos-de-diagnóstico-rápido)
- [7. Restauración completa del entorno Docker] (#7-restauración-completa-del-entorno-docker)

1. El puerto 3306 ya está en uso (conflicto con MariaDB local)

Síntoma

```text

```
Error: Cannot start service mysql: driver failed programming external
connectivity on endpoint erp_mysql:
failed to bind host port 0.0.0.0:3306/tcp: address already in use
```

## Causa

El servicio MariaDB/MySQL nativo de la máquina anfitriona está ocupando el puerto 3306.

## Solución

### Opción A – Detener el servicio local (recomendado si no se necesita)

```
En Linux con systemd
sudo systemctl stop mariadb
sudo systemctl disable mariadb # para que no se inicie automáticamente

En macOS con Homebrew
brew services stop mariadb

Verificar que el puerto quede libre
sudo lsof -i :3306
No debe aparecer ningún proceso
```

Después, levantar Docker:

```
docker-compose up -d
```

### Opción B – Cambiar el puerto expuesto por Docker

Editar `docker-compose.yml`, en el servicio `mysql`:

```
ports:
 - "3307:3306" # mapea el puerto 3307 del host al 3306 del contenedor
```

Actualizar el archivo `.env` del backend:

```
DB_PORT=3307
```

Luego reiniciar Docker:

```
docker-compose down
docker-compose up -d
```

---

## 2. Error 1045 – Access denied for user 'erp\_user'@'localhost'

Síntoma

Al ejecutar `GET /health` la API responde:

```
{
 "status": "error",
 "database": "disconnected",
```

```
"error": "(pymysql.err.OperationalError) (1045, \"Access denied for user
'erp_user'@'localhost' ...")
}
```

## Causa

MySQL rechaza la autenticación porque el usuario `erp_user` no existe con un host compatible, o la contraseña no coincide.

## Solución definitiva

1. Acceder al contenedor MySQL como root:

```
docker exec -it erp_mysql mysql -u root -p
Contraseña: rootpass
```

2. Dentro de MySQL, limpiar y crear correctamente el usuario:

```
-- Eliminar todos los usuarios erp_user existentes
DROP USER IF EXISTS 'erp_user'@'localhost';
DROP USER IF EXISTS 'erp_user'@'%';
DROP USER IF EXISTS 'erp_user'@'127.0.0.1';

-- Crear el usuario accesible desde cualquier host
CREATE USER 'erp_user'@ '%' IDENTIFIED BY 'erp_pass';

-- Otorgar todos los privilegios sobre la base de datos gestion
GRANT ALL PRIVILEGES ON gestion.* TO 'erp_user'@'%';
FLUSH PRIVILEGES;
```

3. (Solo si persiste) Forzar el plugin de autenticación nativo:

```
ALTER USER 'erp_user'@ '%' IDENTIFIED WITH mysql_native_password BY
'erp_pass';
FLUSH PRIVILEGES;
```

4. Verificar que solo exista `'erp_user'@' % '`:

```
SELECT user, host FROM mysql.user WHERE user = 'erp_user';
```

5. Probar la conexión desde el host:

```
mysql -h 127.0.0.1 -P 3306 -u erp_user -p
(o el puerto que estés utilizando)
```

---

### 3. El contenedor erp\_mysql no se considera healthy o tarda en arrancar

#### Síntoma

`docker ps` muestra el estado `unhealthy` o `health: starting` durante mucho tiempo, impidiendo que phpMyAdmin se inicie.

#### Causa

El healthcheck configurado puede ser demasiado estricto o el contenedor necesita más tiempo para inicializarse.

#### Solución

Aumentar los reintentos y el tiempo de espera en el healthcheck del `docker-compose.yml`:

```
healthcheck:
 test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
 interval: 5s
 timeout: 10s
 retries: 20
 start_period: 30s # opcional, tiempo antes de empezar a contar fallos
```

Si aun así falla, se puede temporalmente eliminar la condición `condition: service_healthy` del servicio `phpmyadmin`:

```
depends_on:
 - mysql
```

Luego reiniciar:

```
docker-compose down
docker-compose up -d
```

---

### 4. phpMyAdmin no inicia porque MySQL no está healthy

#### Síntoma

```
ERROR: for phpmyadmin Container "... " is unhealthy.
```

## Solución

1. Verificar el estado de MySQL:

```
docker logs erp_mysql 2>&1 | tail -50
```

Buscar posibles errores de inicialización (scripts en `./database/schema`, permisos, etc.).

2. Si MySQL ya está corriendo y se puede acceder manualmente (paso 5 de la sección 2), pero el healthcheck sigue fallando, ajustar el healthcheck como se indica en la sección 3.

3. Levantar los servicios de forma escalonada:

```
docker-compose up -d mysql redis # solo bases de datos
esperar unos segundos hasta que mysql esté healthy
docker-compose up -d phpmyadmin
```

---

## 5. Cambio permanente de puerto para evitar conflictos con MariaDB

Para no depender de detener el servicio local cada vez, se recomienda cambiar el puerto expuesto de Docker de forma definitiva.

### Procedimiento

1. Editar `docker-compose.yml`:

```
services:
 mysql:
 ports:
 - "3307:3306" # el puerto externo puede ser cualquier número libre
 phpmyadmin:
 environment:
 PMA_HOST: mysql
 PMA_PORT: 3306 # se mantiene el puerto interno del contenedor
```

2. Actualizar el archivo `.env` del backend:

```
DB_PORT=3307
```

### 3. Recrear los contenedores:

```
docker-compose down
docker-compose up -d
```

### 4. Verificar la conexión:

```
mysql -h 127.0.0.1 -P 3307 -u erp_user -p
```

Este cambio es persistente y evita cualquier conflicto futuro.

---

## 6. Comandos de diagnóstico rápido

| Propósito                      | Comando                                                          |
|--------------------------------|------------------------------------------------------------------|
| Ver puertos ocupados           | <code>sudo lsof -i :3306</code>                                  |
| Ver estado de los contenedores | <code>docker-compose ps</code>                                   |
| Logs recientes de MySQL        | <code>docker logs erp_mysql --tail 50</code>                     |
| Entrar a MySQL como root       | <code>docker exec -it erp_mysql mysql -u root -p</code>          |
| Probar conexión desde el host  | <code>mysql -h 127.0.0.1 -P &lt;puerto&gt; -u erp_user -p</code> |
| Listar usuarios MySQL          | <code>SELECT user, host FROM mysql.user;</code>                  |

---

## 7. Restauración completa del entorno Docker

Cuando se necesite empezar desde cero (se pierden todos los datos):

```
En la carpeta del proyecto
docker-compose down -v # elimina contenedores, redes y volúmenes

Si quedan redes colgantes
docker network prune -f

Opcional: eliminar imágenes para forzar su descarga
docker rmi mysql:8.0 phpmyadmin/phpmyadmin redis:7-alpine

Volver a construir
docker-compose up -d
```

Recuerda luego revisar el usuario `erp_user` (sección 2) y el puerto (sección 1/5), sobre todo si el volumen se recreó.

---

## Nota adicional sobre los scripts de inicialización

La carpeta `./database/schema` se monta en `/docker-entrypoint-initdb.d` y se ejecuta **solo la primera vez que se crea el volumen**. Si se necesita recrear la base de datos sin perder otros datos, se puede eliminar el volumen de MySQL:

```
docker-compose down
docker volume rm proyecto-erp_mysql_data # nombre exacto con docker
volume ls
docker-compose up -d
```

---

Última actualización: [fecha]

Proyecto ERP — Documentación de configuración

Guarda este contenido en un archivo llamado ``solucion-docker-mysql.md`` en la raíz de tu proyecto. Así tendrás siempre a mano la solución para estos problemas recurrentes. Si me dices qué editor usas, te puedo sugerir cómo guardarlo.